East West University

Department of Computer Science and Engineering

Semester: Fall 2023

Course Number: CSE345

Section: 03

Course Title: Digital Logic Design

Group Number: 01

Project Number: Project-1

Group Members:

i) 2021-2-60-063 (MD. Ismail Bhuiyan)
ii) 2020-1-60-172 (Tanvir Ahmed)
iii) 2020-2-60-123 (Rihan Mahmud)

Submitted To,

Musharrat Khan

Senior Lecturer

Department of Computer Science and Engineering

East West University

1. Problem Statement:

A 4-bit code converter for odd parity converts the 4-bit input to 5-bit output so that odd parity is ensured.

2. Design Details:

Let's say 4-bit inputs are A3, A2, A1, A0 and 5-bit outputs are O3, O2, O1, O0, and P where, O3, O2, O1, O0 outputs are equivalent to the inputs A3, A2, A1, and A0 respectively and P is the parity bit.

Truth Table:

Input				Output				
A3	**A2**	**A1**	**A0**	**O3**	**O2**	**O1**	**O0**	**P**
0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1	0
0	0	1	0	0	0	1	0	0
0	0	1	1	0	0	1	1	1
0	1	0	0	0	1	0	0	0
0	1	0	1	0	1	0	1	1
0	1	1	0	0	1	1	0	1
0	1	1	1	0	1	1	1	0
1	0	0	0	1	0	0	0	0
1	0	0	1	1	0	0	1	1
1	0	1	0	1	0	1	0	1
1	0	1	1	1	0	1	1	0
1	1	0	0	1	1	0	0	1
1	1	0	1	1	1	0	1	0
1	1	1	0	1	1	1	0	0
1	1	1	1	1	1	1	1	1

Here,

O3 = A3;

O2 = A2;

O1 = A1;

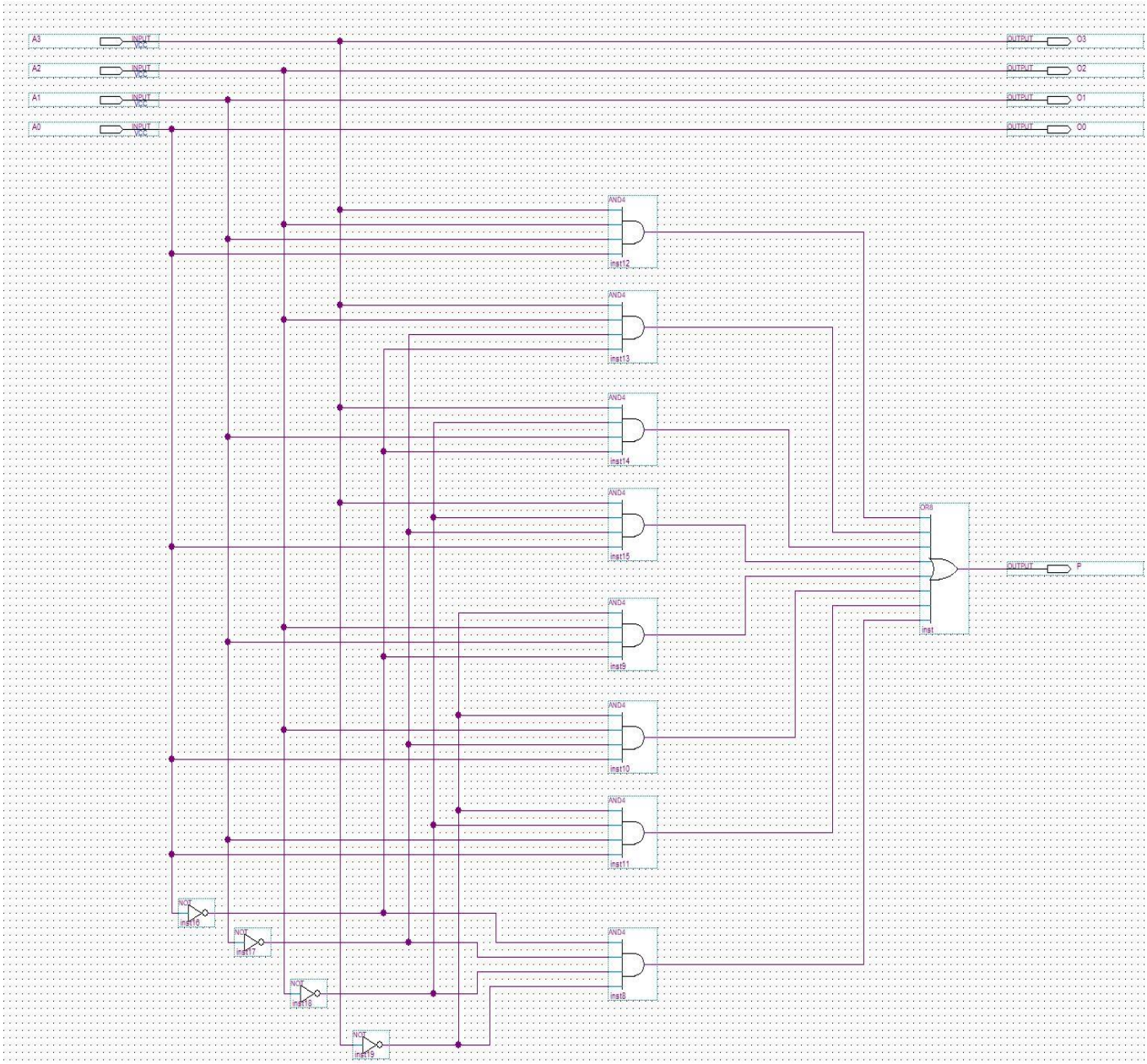
O0 = A0;

For Parity bit P, K-map:

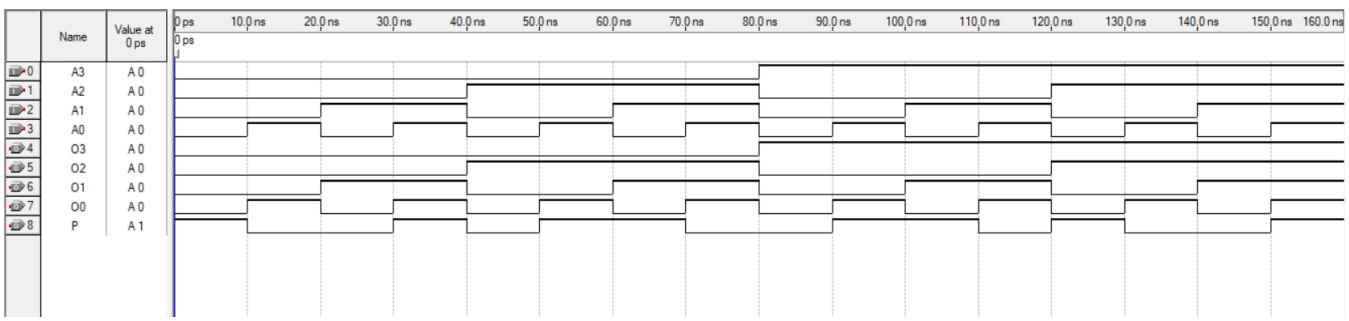
A3 A2 \ A1 A0	00	01	11	10
00	1		1	
01		1		1
11	1		1	
10		1		1

$\therefore P = A3'A2'A1'A0' + A3'A2'A1A0 + A3'A2A1'A0 + A3'A2A1A0' + A3A2'A1'A0 + A3A2'A1A0' + A3A2A1'A0' + A3A2A1A0$

3. Logic Diagram:



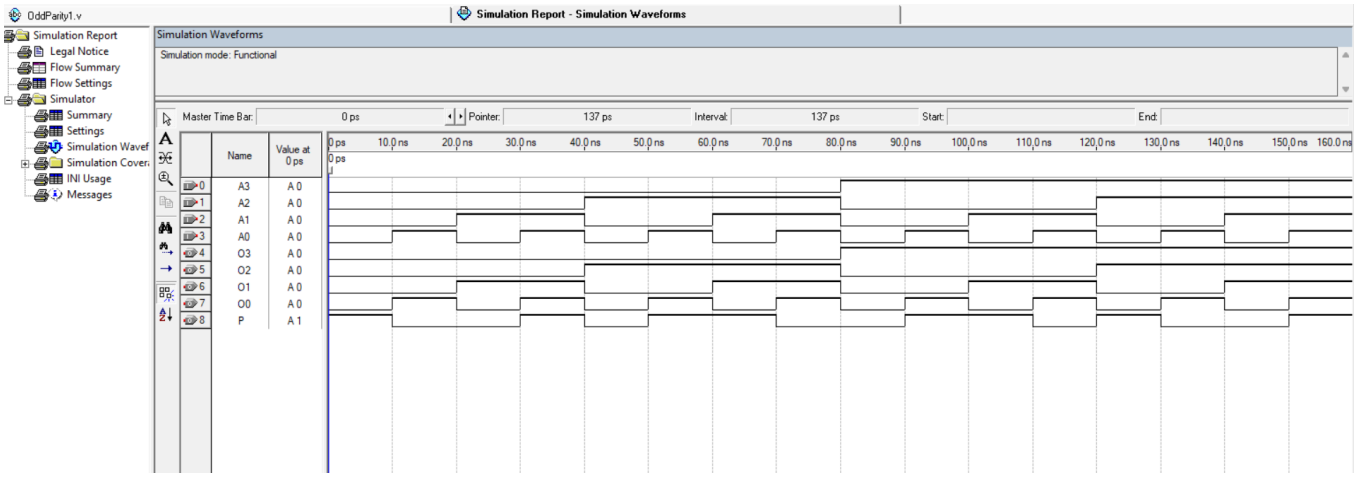
Simulation Results:



4. Structural Verilog Code:

```
module OddParity1(input A3,A2,A1,A0,output O3,O2,O1,O0,P);
        wire w1,w2,w3,w4,w5,w6,w7,w8;
        and g1(w1,~A3,~A2,~A1,~A0),
                g2(w2,~A3,~A2,A1,A0),
                g3(w3,~A3,A2,~A1,A0),
                g4(w4,~A3,A2,A1,~A0),
                g5(w5,A3,~A2,~A1,A0),
                g6(w6,A3,~A2,A1,~A0),
                g7(w7,A3,A2,~A1,~A0),
                g8(w8,A3,A2,A1,A0);
        or      g9(P,w1,w2,w3,w4,w5,w6,w7,w8),
                g10(O0,A0),
                g11(O1,A1),
                g12(O2,A2),
                g13(O3,A3);
endmodule
```

Simulation Results:

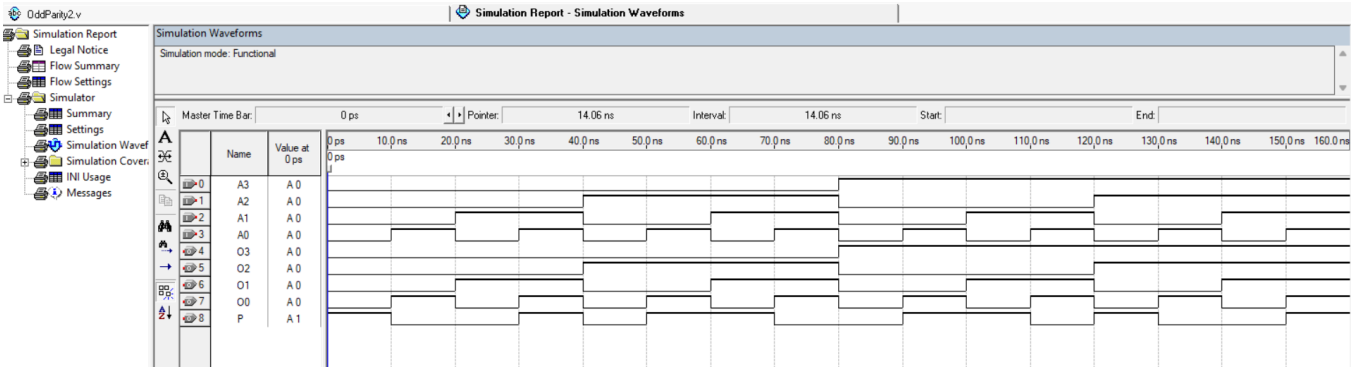


5. Behavioral Verilog Code:

5.1. Procedural Model:

```
module OddParity2(input A3,A2,A1,A0, output reg O3,O2,O1,O0,P);
        always@(A3,A2,A1,A0)
                begin
                        O3=A3;
                        O2=A2;
                        O1=A1;
                        O0=A0;
                        P=0;
                        if(~A3&~A2&~A1&~A0) P=1;
                        if(~A3&~A2&A1&A0) P=1;
                        if(~A3&A2&~A1&A0) P=1;
                        if(~A3&A2&A1&~A0) P=1;
                        if(A3&~A2&~A1&A0) P=1;
                        if(A3&~A2&A1&~A0) P=1;
                        if(A3&A2&~A1&~A0) P=1;
                        if(A3&A2&A1&A0) P=1;
                end
endmodule
```

Simulation Results:



5.2. Continuous Assign Statement:

```
module OddParity3(input A3,A2,A1,A0, output O3,O2,O1,O0,P);

        assign O3=A3;

        assign O2=A2;

        assign O1=A1;

        assign O0=A0;

        assign P = (~A3&~A2&~A1&~A0) | (~A3&~A2&A1&A0) | (~A3&A2&~A1&A0) |
        (~A3&A2&A1&~A0) | (A3&~A2&~A1&A0) | (A3&~A2&A1&~A0) |
        (A3&A2&~A1&~A0) | (A3&A2&A1&A0);

endmodule
```

Simulation Results: